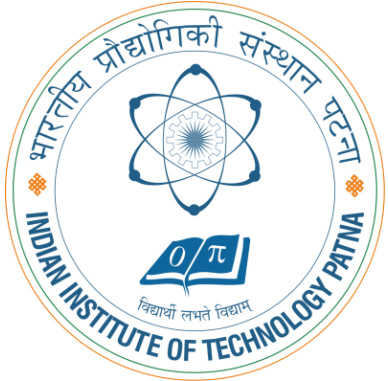


CS5102: FOUNDATIONS OF COMPUTER SYSTEMS

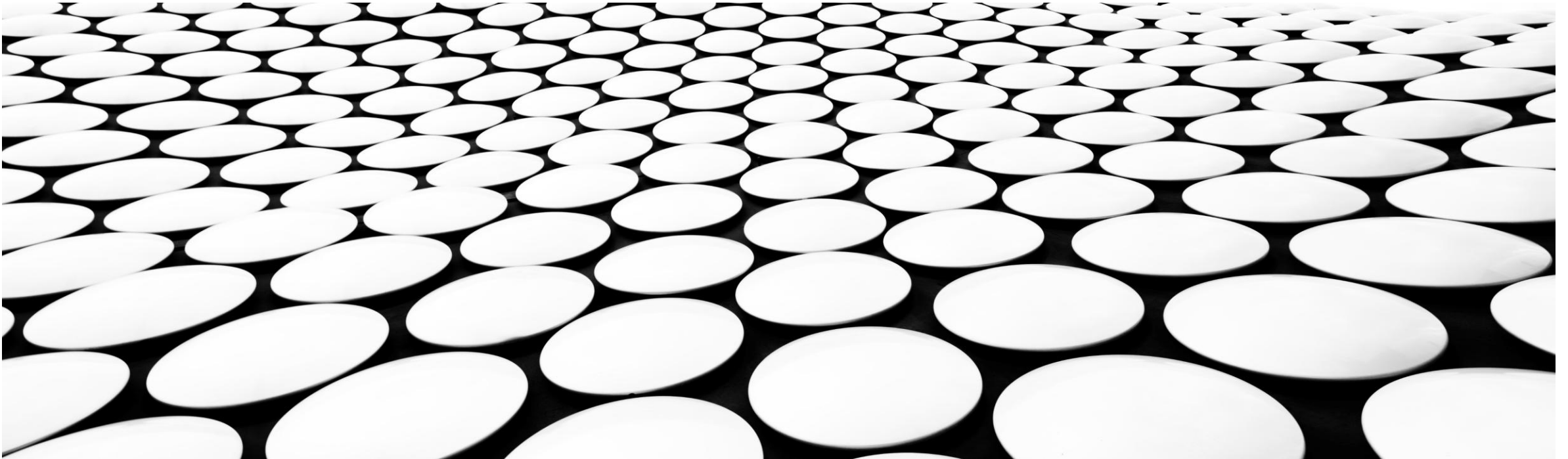


TOPIC-19: CPU SCHEDULING

DR. ARIJIT ROY

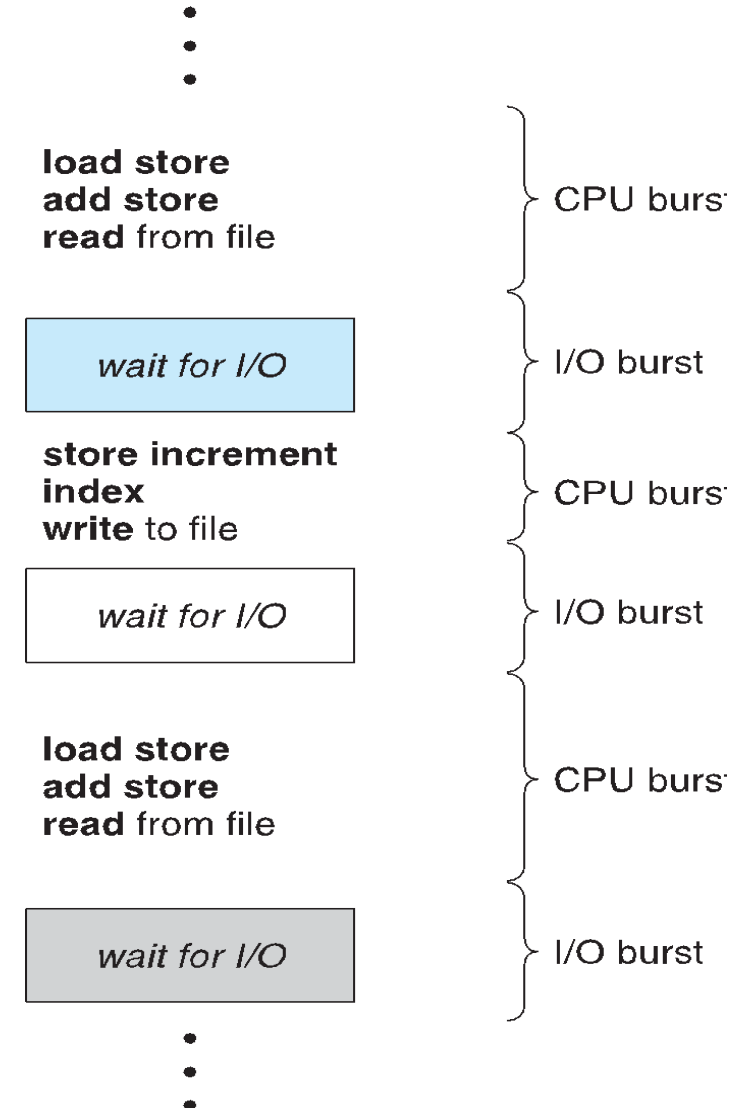
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY PATNA



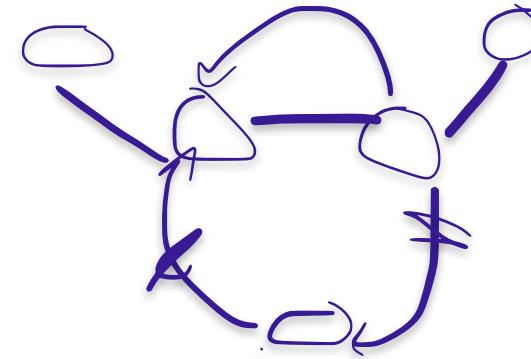
BASIC CONCEPTS

- **Maximum CPU utilization** obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- **CPU burst** followed by **I/O burst**
- CPU burst distribution is of main concern



CPU SCHEDULER

- **Short-term scheduler** selects from among the processes in ready queue, and allocates the CPU to one of them
 - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state
 2. Switches from running to ready state
 3. Switches from waiting to ready
 4. Terminates
- Scheduling under 1 and 4 is **nonpreemptive**
- All other scheduling is **preemptive**
 - Consider access to shared data
 - Consider preemption while in kernel mode
 - Consider interrupts occurring during crucial OS activities



DISPATCHER

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

SCHEDULING CRITERIA

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process (submission to completion)
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

SCHEDULING ALGORITHM OPTIMIZATION CRITERIA



- Max CPU utilization ✓
- Max throughput ✓
- Min turnaround time ✓
- Min waiting time
- Min response time

FIRST- COME, FIRST-SERVED (FCFS) SCHEDULING



Process

Burst Time

Time the process will take to complete its execution

Pre-emptive

P_1

24 ✓

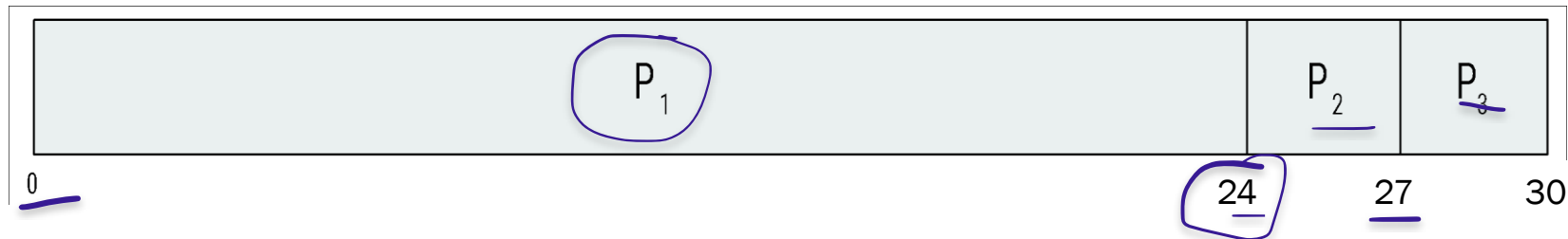
P_2

3 ✓

P_3

3 ✓

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS SCHEDULING (CONT.)

Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$



FCFS SCHEDULING (CONT.)

Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case
- **Convoy effect** - short process behind long process
 - Consider one CPU-bound and many I/O-bound processes

SHORTEST-JOB-FIRST (SJF) SCHEDULING

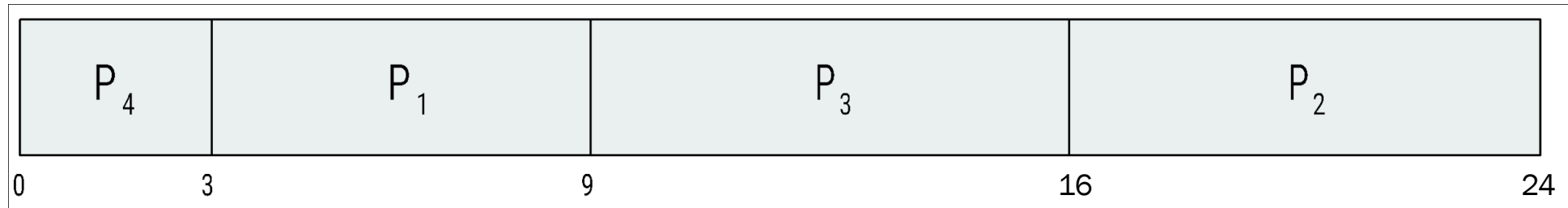
Non-Pre-emptive

- Associate with each process the length of its next CPU burst
 - Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
 - The difficulty is knowing the length of the next CPU request
 - Could ask the user

EXAMPLE OF SJF

<u>Process</u>	<u>Burst Time</u>
<u>P_1</u>	<u>6</u>
<u>P_2</u>	8
<u>P_3</u>	7
<u>P_4</u>	<u>3</u>

- SJF scheduling chart



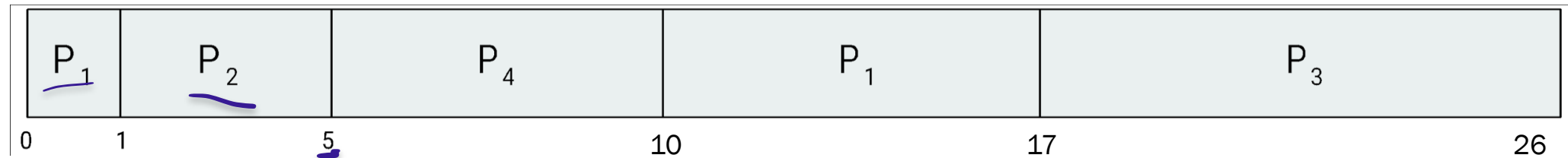
- Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7$

EXAMPLE OF SHORTEST-REMAINING-TIME-FIRST

- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0 ✓	<u>8</u> ✓ 7
P_2	<u>1</u> ✓	4 ✓ 4
P_3	2 ✓	<u>9</u> ✓
P_4	3 ✓	<u>5</u> ✓

- Preemptive SJF Gantt Chart*



- Average waiting time = $[(10-1)+(1-1)+(17-2)+5-3]/4 = 26/4 = \underline{6.5 \text{ msec}}$

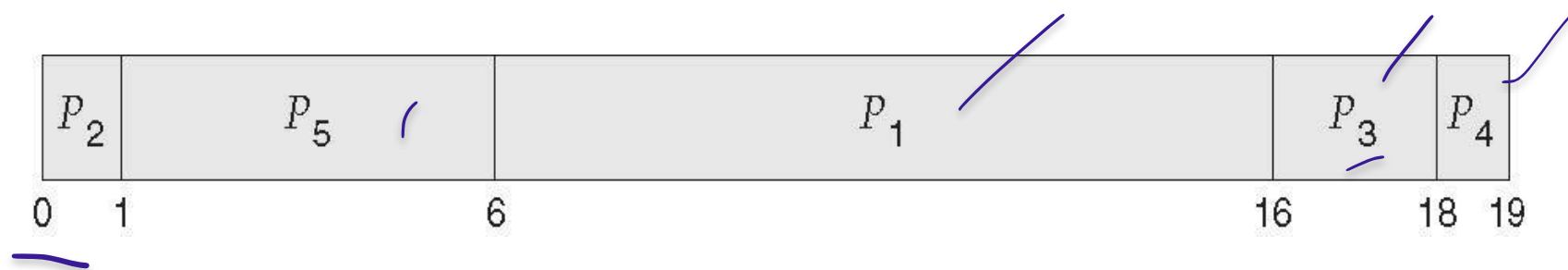
PRIORITY SCHEDULING

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer = highest priority)
 - Preemptive
 - Nonpreemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time

EXAMPLE OF PRIORITY SCHEDULING

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3 ✓
P_2	1	1 ✓
P_3	2	4 ✓
<u>P_4</u>	1	<u>5</u> ✓
P_5	5	<u>2</u>

- Priority scheduling Gantt Chart



- Average waiting time = 8.2 msec

PROBLEM?



Starvation – low priority processes may never execute

Aging – as time progresses increase the priority of the process

ROUND ROBIN (RR)



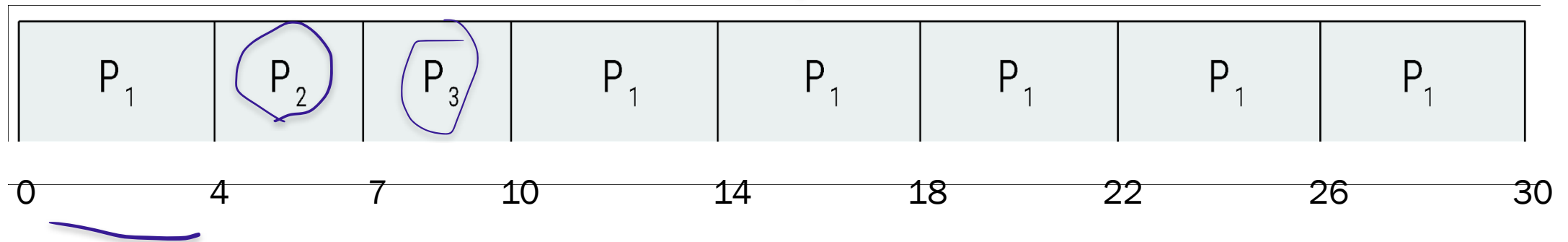
- Each process gets a small unit of CPU time (**time quantum q**), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Timer interrupts every quantum to schedule next process
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high

EXAMPLE OF RR WITH TIME QUANTUM = 4

Process Burst Time

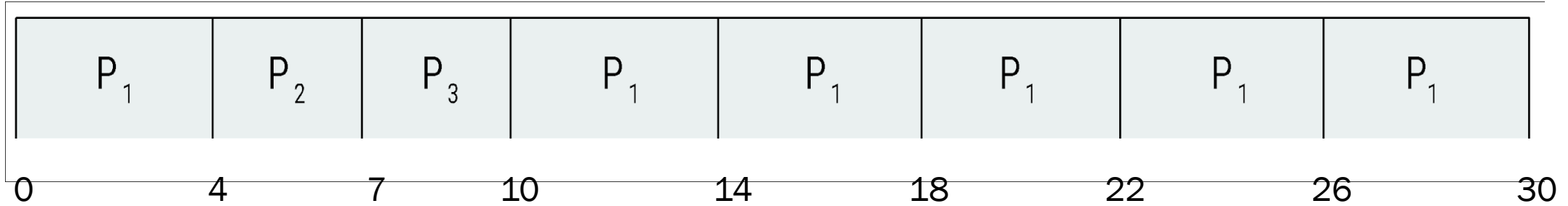
<u>P_1</u>	<u>24</u>
<u>P_2</u>	<u>3</u>
<u>P_3</u>	<u>3</u>

- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better **response**
- q should be large compared to context switch time
- q usually 10ms to 100ms, context switch < 10 usec

TURNAROUND AND WAITING TIMES



✓ **Turnaround time** = completion time – Arrival time ✓
Waiting time = turnaround time – burst time

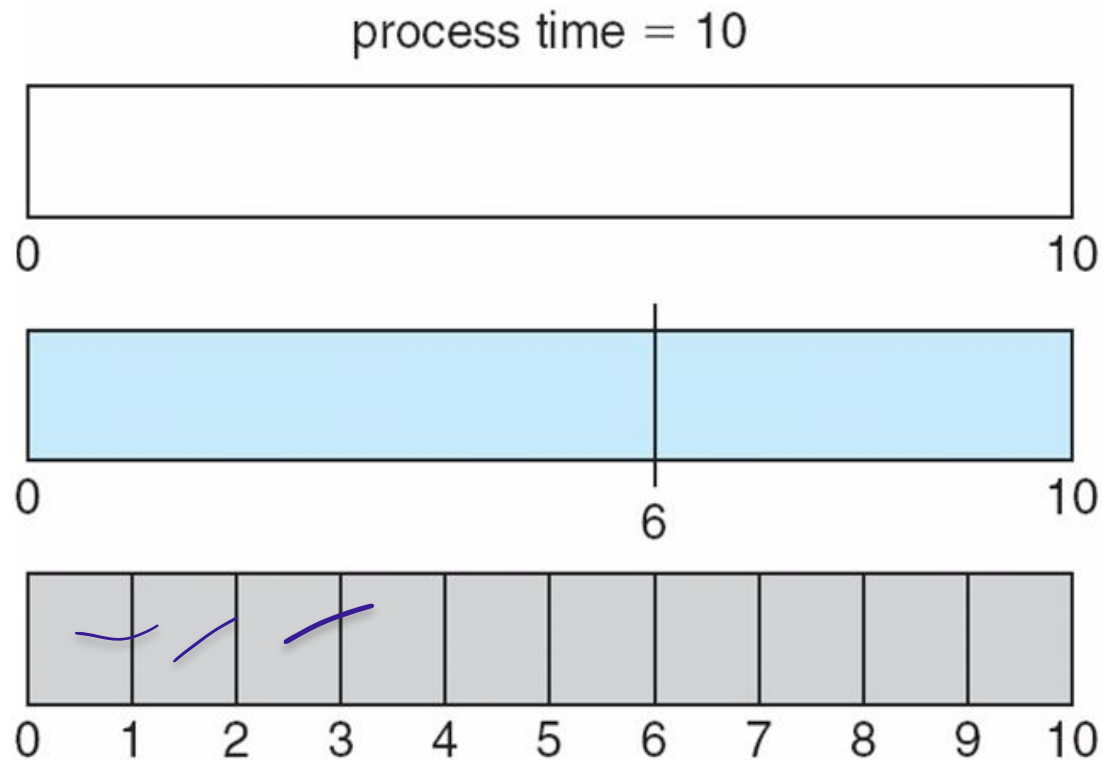
Process ID	Completion time	Turnaround time	Waiting time
P ₁	30 ✓	$30 - 0 = 30$	$30 - 24 = 6$
P ₂	7 ✓	$7 - 0 = 7$	$7 - 3 = 4$
P ₃	10	$10 - 0 = 10$	$10 - 3 = 7$

Process	Burst Time
P ₁	24
P ₂	3
P ₃	3

Average Turnaround time = $47/3 = 15.66$ ✓

Average Waiting time = $17/3 = 5.66$

TIME QUANTUM AND CONTEXT SWITCH TIME



quantum

12

6

1

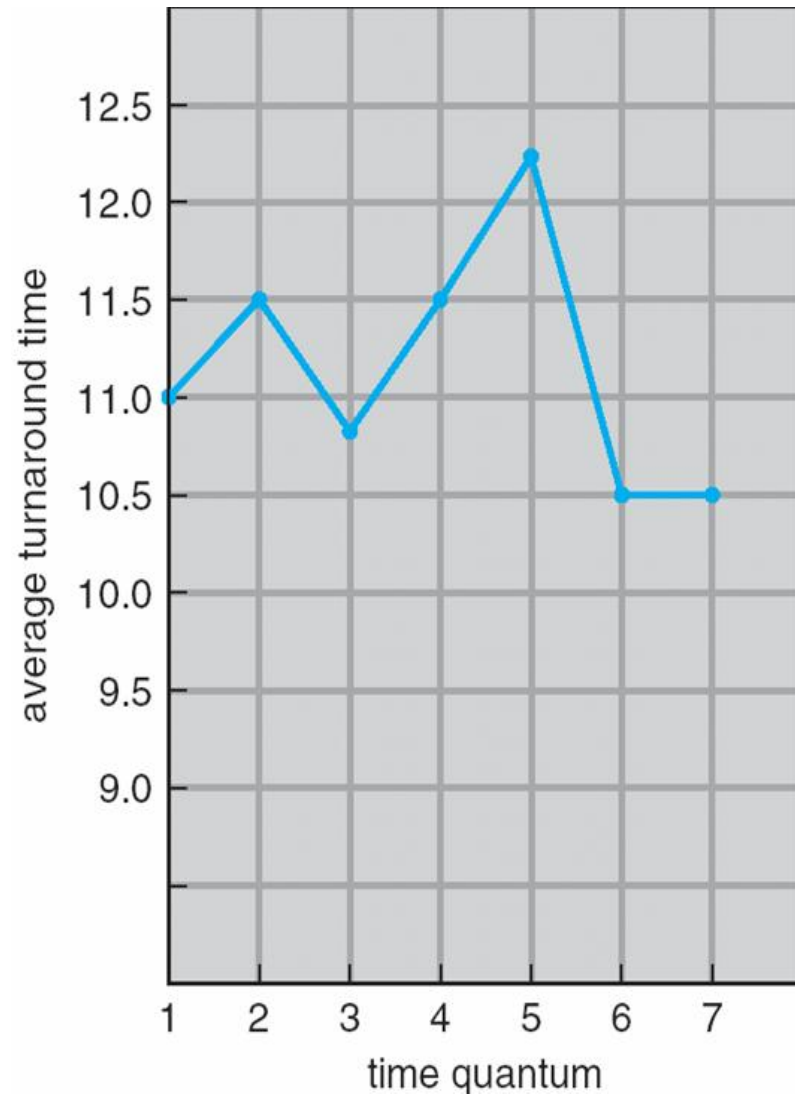
context
switches

0

1

9

TURNAROUND TIME VARIES WITH THE TIME QUANTUM

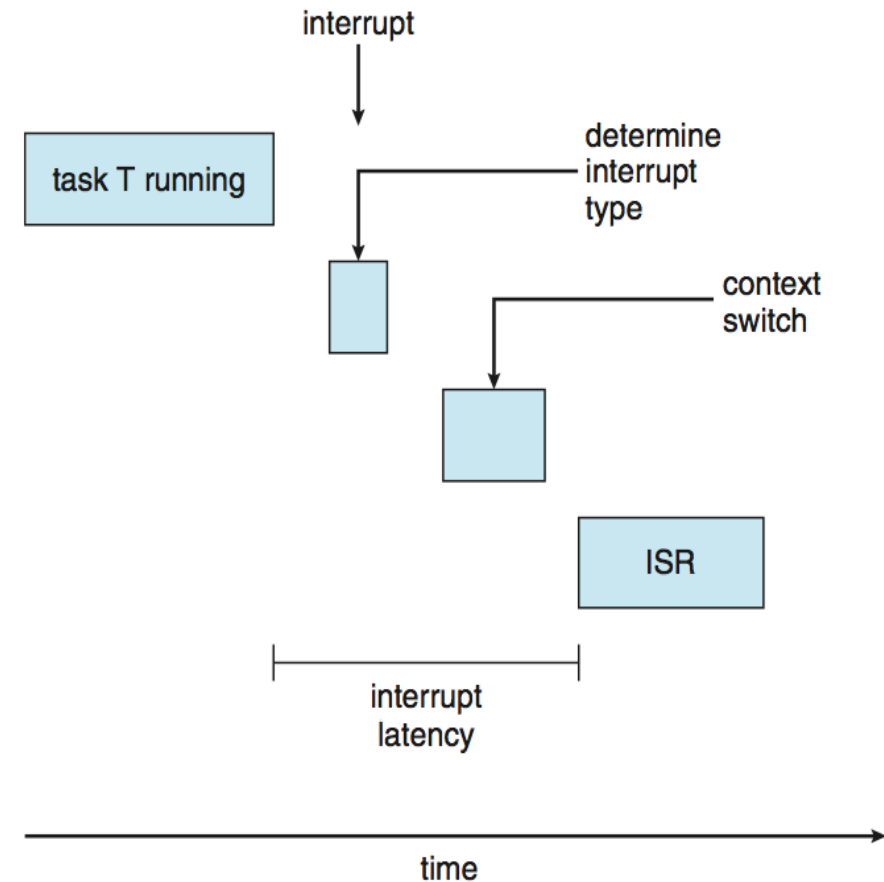


process	time
P_1	6
P_2	3
P_3	1
P_4	7

80% of CPU bursts should be shorter than q

REAL-TIME CPU SCHEDULING

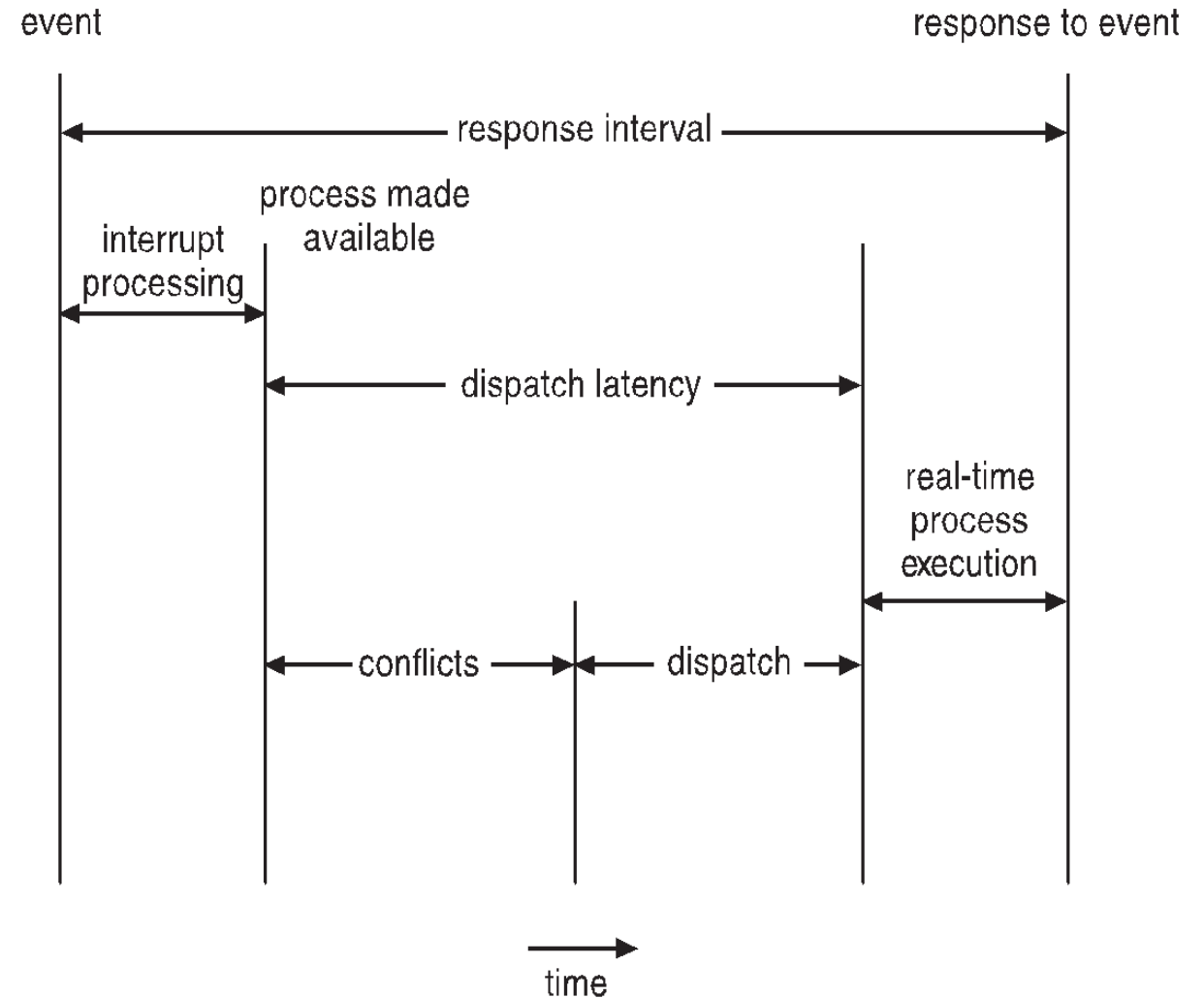
- Can present obvious challenges
- **Soft real-time systems** – no guarantee as to when critical real-time process will be scheduled
- **Hard real-time systems** – task must be serviced by its deadline
- Two types of latencies affect performance
 1. Interrupt latency – time from arrival of interrupt to start of routine that services interrupt
 2. Dispatch latency – time for schedule to take current process off CPU and switch to another



REAL-TIME CPU SCHEDULING (CONT.)

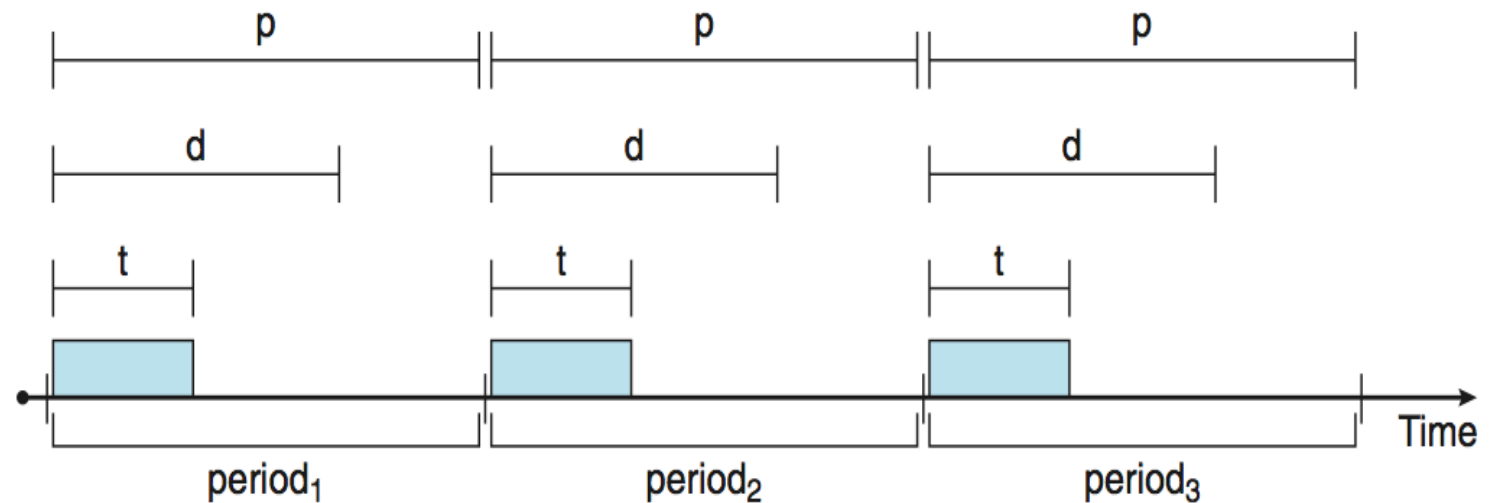


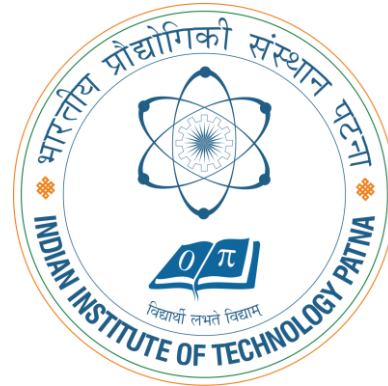
- Conflict phase of dispatch latency:
 1. Preemption of any process running in kernel mode
 2. Release by low-priority process of resources needed by high-priority processes



PRIORITY-BASED SCHEDULING

- For real-time scheduling, scheduler must support preemptive, priority-based scheduling
 - But only guarantees soft real-time
- For hard real-time must also provide ability to meet deadlines
- Processes have new characteristics: **periodic** ones require CPU at constant intervals
 - Has processing time t , deadline d , period p
 - $0 \leq t \leq d \leq p$
 - **Rate** of periodic task is $1/p$





THANK YOU!